

Variables and Datastructures

1. What are Variables?
2. What are you?
3. Nicer print outs
4. Converting variables
5. A bit mathematics
6. Changing the values of Variables
7. What are datastructures?
8. Lists
9. Dictionaries
10. Tuples
11. Sets
12. Nesting datastructures
13. Definitions

What are variables?

- Variables are named containers that contain data or values of a specific type
- They are placeholders for actual values, that means we use the name of the variable to access the value stored in it
- Consist of three elements:
 - Variable type
 - Concrete value
 - Name of the variable

- Prior to using variables they have to be made accessible to the program, this is called declaring a variable
- Here we define the name, type and the value of our variable
- The name should be descriptive so that we always know what this variable represents later on
- Generally we declare variables at the beginning of our script, so that we can use them from this point on in our algorithm

- Some programming languages allow declaring variables without specifying a type during this process however in these cases we always need to define a value
- Other languages allow you to declare a variable without a value, as long as you define the type
- In Python we always have to declare/define a variable by specifying a value we want to save in the variable

```
first_name = „Bill“      age = 55      distance = 20.35
```

- As mentioned the value of a variable is a part of it and the association of a value with a variable is called Assigning a specific value to a variable
- Normally done with the “=” operator
- Not to be mixed up with the mathematical equalness
- Variable is always on the left and the value is on the right side of the equal sign
- Assigning a value of a wrong type can lead to compiler errors or unwanted results

- The declaration of a variable refers to specifying the name and datatype of a variable
- Assignment refers to associating a value with a variable
- The definition of a variable refers to declaring a variable and assigning a value to it at the same time

- If the program is executed a simple text replacement is taking place – every instance of the variable name is replaced with the specified value and the source code is executed with these values

Data type	Description	Memory	Values
byte	Whole positive numbers	8bit	0 to 255
sbyte	Whole numbers with signs	8bit	-128 to 127
short	Whole numbers with signs	16bit	-32,768 to 32,767
int	Whole numbers with signs	32bit	-2,147,483,648 to 2,147,483,647
long	Whole numbers with signs	64bit	-92,233,727,203,685,477,508 to 92,233,727,203,685,477,507
float	Floating-point numbers with simple accuracy	32bit	-3.402823e38 to 3.402823e38
double	Floating-point numbers with double accuracy	64bit	-1.79769313486232e308 to 1.79769313486232e308
bool	Truth value (Yes, No)	8bit	true / false
char	One symbol	16bit	One unicode symbol
string	Text		Random text

- In Python we always need to declare and assign variables at the same time with the help of the equal sign (=), thus we define variables
- Per definition the variable always stands on the left side of the = and the value to be stored on the right side
- The line `first_name = "Bill"` can be interpreted as put the value "Bill" in the Box first_name
 - That's how you store strings
- The Line `number1 = 432` means put the value 432 into the box number1
 - That's how you store numbers
- This process of declaring and assigning variables without specifying a data type in the process is called duck typing

- In Python there are several conventions for naming variables:
 - ▶ be all lowercase
 - ▶ be in general more descriptive than variable names such as “a” or “b”
 - ▶ with individual words separated by underscores as necessary to improve readability
 - ▶ avoid names like int, str, list and so on that could also be used as built-in functions, instead use names like number1, input_list etc.

hello_world = "Hello World"

➔ hello_world contains "Hello World"

number = 10

➔ number contains 10

decimal = 7.9

➔ decimal contains 7.9

bool_value = True

➔ bool_value contains True

print(hello_world)

➔ compiler reads print("Hello World")

print(number)

➔ compiler reads print(10)

print(decimal)

➔ compiler reads print(7.9)

print(bool_value)

➔ compiler reads print(True)

- In informatics the name variables describes something that is different from the mathematical meaning of this term:
 - ▶ Mathematics: an unknown value you need to find out
 - ▶ Informatics: a placeholder for a known value or result

- As a constant that we can reuse over and over again for different calculations
- As a way to save intermediate results, to use them in later calculations / analysis
- In form of data structures, to collect results of multiple calculations and run statistical analysis on them
- To control how loops behave (later)
- To form connections to webserver (biopython), databases (mysql) or text files
- ...

- `print()` is a so-called built-in function that fulfills a very specific task, here displaying something in the terminal
- Python has many of these built-in functions we can use to help us with our programs
- We use them by simply writing the name of the function and then add brackets `()` to the end of it, this is the call of the function
- In these brackets we write the things we want the function to be working with, e.g. a variable or a string
- The things written in brackets are called arguments of the function and different functions have different conditions for their arguments

- Over the course of time you will learn about many different built-in functions and their arguments
- It might be a good idea, especially for you as beginners to create some form of leaflet containing an overview of these built-in functions
- On there you could write the function, its uses, the output and what arguments it needs

- As mentioned earlier `print()` displays whatever we hand over as argument in the terminal
- This is the only use of `print` and still it's one of the most-used if not the most-used built-in function, since it allows you to track your program
- `print()` is the only way of displaying / looking at things during the execution of your program!

- Write a small Python program that declares and assigns (defines) at least the different Variables listed below:
 - ▶ age, salary, hours_per_day as integers
 - ▶ volume, mass, time as floats
 - ▶ first_name, last_name, workplace as strings
 - ▶ has_studied, has_phd as booleans
- Use print to show the values of all variables in the terminal

What are you?

- You learned how to declare and assign (define) variables
- To make sure we can use the needed operations with our variables we need to check whether or not they have the right data type
- We can always look up this data type of our variables with the `type()` function which returns the corresponding data type (or type of data structure)
- For example to look up the data type of the variable `number1` we can use:

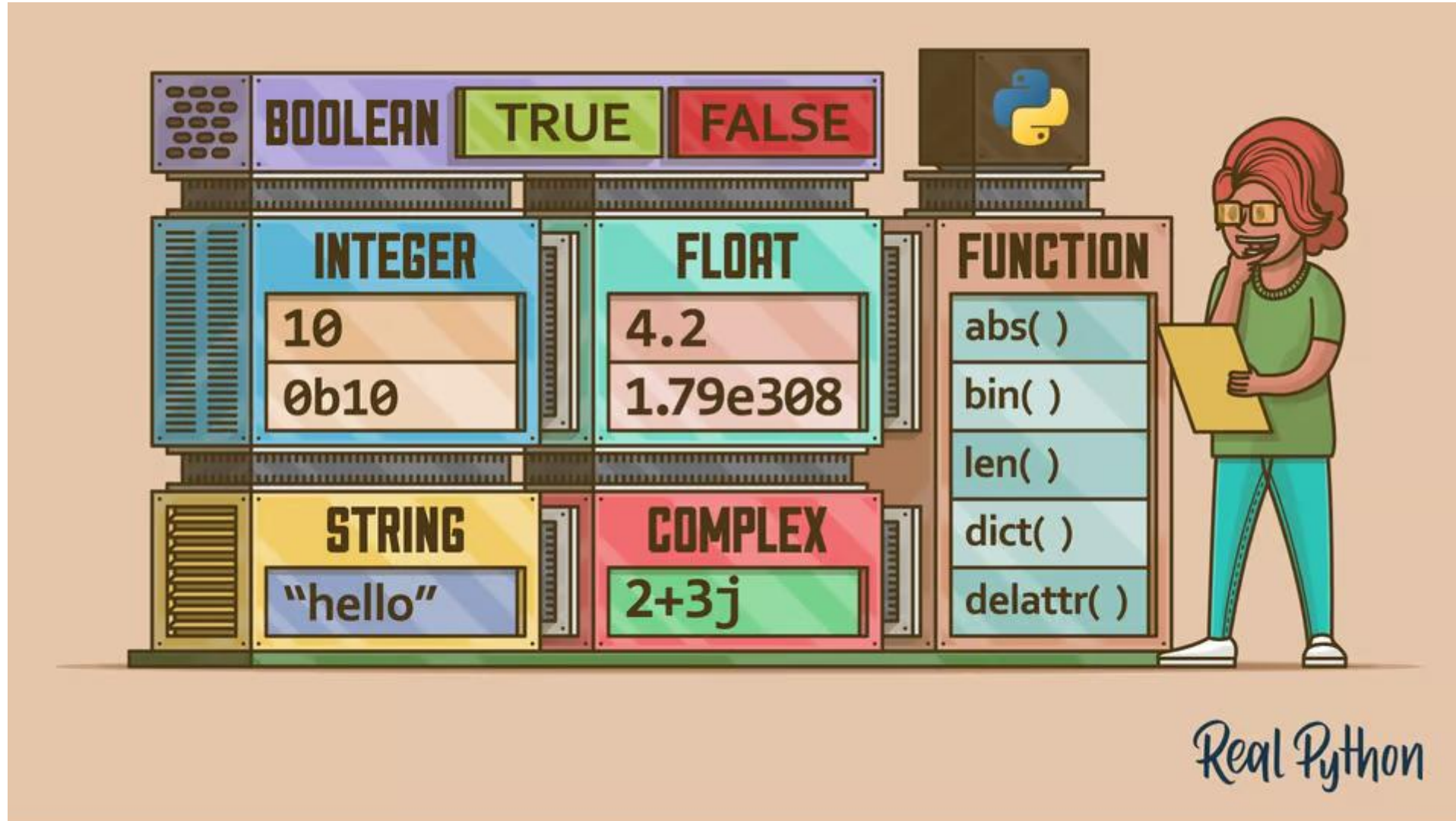
```
type(number1)
```

- However since this only determines the data type we should use `type()` in combination with `print()` to see the result in the terminal:

```
print(type(number1))
```

- Don't be confused about returns like `<class 'int'>` because everything in Python is a class and we will learn more about that when we talk about object orientated programming
- For now the important part of the return is the 'int' which tells us our variable is of type integer
- The `<class 'int'>` is the output of the function type and could be assigned to a variable
- However in nearly 99.9% of cases you use type either in combination with print or in an if-statement (more about this later)
- `type()` only takes one argument

- CQ** Beratung
+ Bildung



See [13 – 15] for more information regarding binary notations like 0b10

- Define at least seven different variables of different types:
 - Integer
 - Float
 - String
 - Bool
- Print out their value as well as the corresponding type of them

Nicer print outs

- We saw how you can use print, to display strings in the terminal
- You can also use print to display the values of variables in the terminal
- Furthermore you can combine strings and variables in a print command to generate print outs that contain more information
- This can be done in at least two ways:
 - Using commas (i.e. multiple arguments)
 - Using f-Strings
- These kind of print outs are useful to analyze how the value of your variables change over the course of a script and to see whether or not your script works as intended

- The built-in function print can take more than just one argument
- There is no upper limit for the number of arguments we can hand over to print
- We have to comma separate these arguments so that print recognizes them:

```
print("Hello World", number1)
```

```
print("My Name is", first_name, "and I'm", age, "years old")
```

- Per default print enters a space between each argument before printing the result out in the terminal

- Also called “formatted string literals,”
- Are strings that have an f at the beginning and curly brackets containing different expressions that will be replaced with their values:

```
print(f"My Name is {first_name} and I'm {age} years old")
```

- So in the example above the variables `first_name` and `age` are replaced by their values and these values are converted to a string, that means text and than everything is displayed together in the terminal

- f-Strings are evaluated at run time, that means we can put any expressions in them:

```
print(f"The result is {2 * 37} meters")
```

- In the example above Python first calculates $2 \cdot 37$ (the result being 74) and then converts this number into a string and displays everything afterwards in the terminal

- Write a script that defines the variables `full_name` (str), `age` (int), `hours_per_day` (int), `salary` (int), `time`(float), `workplace` (str), `lesson` (str) and `has_studied` (bool)
- Use the function `print` to generate displays in the shell as shown below (Variable names in bold):
 - ▶ Hello my name is **full_name** and I work at **workplace**
 - ▶ I am **age** years old and I work **hours_per_day** for a salary of **salary**
 - ▶ Currently **time** hours of today's lesson about **lesson** are over
 - ▶ Did the participant **full_name** study? **has_studied**
- Create each statement with f-strings as well as a `print` command that uses multiple arguments

Converting Variables

- You can convert values of variables from one data type to another
- This process is called casting, but this can only be done if the value you want to cast to another data type fulfills the right conditions:
 - ▶ `String1 = „Hello“` → Can't be casted to int or float
 - ▶ `String1 = „5“` → Can be casted to int and float
 - ▶ `String1 = „7.5“` → Can be casted only to float
- There are three built-in functions for casting in python that belong to one of our three data types each:
 - ▶ `int()`, `float()` and `str()`
- All three can take only one argument

- All three functions produce a new value that is the converted version of the argument
- Therefore you have to use them in combination with a variable assignment:

```
string1 = "10.56"
```

```
number1 = float(string1)      ➔ number1 has the value 10.56
```

- The value and type of string1 in the example above is not changed during this process, since the casted value was assigned to number1

- We can convert only strings that match the integer data type to integer
 - ▶ “100” is castable
 - ▶ “20.58” is NOT castable
 - ▶ “Hello” is NOT castable
- We can convert floats to integers, but we have to keep in mind that Python does NOT round in this case, instead it cuts off all decimals
 - ▶ casting 25.78 returns an integer of 25

- We can always cast integers to floats, since all we need to do is to add a .0 at the end
 - casting 5 to float results in 5.0
- As long as strings contain only numbers and one decimal point at maximum we can always cast them to float as well
 - casting “10” to float results in 10.0
 - casting “25.89” to float results in 25.89
 - “Hello” can NOT be casted

- Be careful when working with floats, since Python uses a binary representation of floating point numbers [9] it can happen that we get back something like 10.000000000001 when casting 10 to a float
- In these cases using round() can be useful (next slide)

- the built-in function round() allows you to round numerical values (integers or floats)
- round() takes 1 or 2 arguments depending on what you want to achieve
- if you use round() with just one argument you get back the nearest integer value:

```
round(10.567)
```

➔ returns 11

- if you use round() with two arguments you can specify the amount of decimals as second argument
- In these cases you will always get back a floating point number:

```
round(10.567, 0)
```

➔ returns 11.0

```
round(10.567, 1)
```

➔ returns 10.6

- Casting anything to string works just fine
- the casting function `str()` is one of the most robust built-in functions in the Python installation
- When using `str()` we can assume that Python simply puts quotation marks around whatever we hand over as argument to it

- Write a script that defines at least five variables with the values “500”, “48.86”, 50.97, 10, “-15”
- Print them out
- Afterwards use the casting functions `int()`, `float()` and `str()` to change the datatypes of your variables
- Make all possible conversions, e.g. the value “500” can be converted to integer and float
- Don’t forget to print out the type of your variable before and after casting it

A bit mathematics

- There are six different basic mathematical operations that can be used for numbers in python:

▶ Addition:	+	➔ 1 + 2	➔ 3
▶ Subtraction:	-	➔ 4 - 2	➔ 2
▶ Multiplication:	*	➔ 3 * 5	➔ 15
▶ Division:	/	➔ 14 / 3	➔ 4.6666
▶ Integer Division:	//	➔ 14 // 3	➔ 4
▶ Power:	**	➔ 5 ** 2	➔ 25

- We can use these operations to assign their result to a new variable:

```
sum1 = 5 + 2
```

- We are also able to use these operations in combination with variables:

```
number1 = 10
```

```
result1 = number1 + 5
```

- Last but not least we are able to combine all of this:

```
number1 = 10
```

```
number2 = 3
```

```
result2 = number1 * number2
```

- The data type of the result of these operations depends on the operation itself and on the data type of the values you work with
- The table below shows these dependencies
- Generally speaking as soon as you have one floating point number anywhere inside your calculation the result will also be a floating point number

value 1	value 2	operation	result
int	int	+ & - & * & // & **	int
int	int	/	float
int	float	+ & - & * & / & // & **	float

- Write a program that shows defines three integer variables (num1, num2 & num3) as well as two floating point variables (num4 & num5)
- Use these variables to calculate the following:
 - ▶ $\text{num1} + \text{num2} + \text{num3}$
 - ▶ $\text{num1} * \text{num2} + \text{num3}$
 - ▶ $\text{num1} * (\text{num2} + \text{num3})$
 - ▶ $\text{num1} / \text{num2}$
 - ▶ $\text{num4} / \text{num5}$
 - ▶ $\text{num5} - \text{num4}$
 - ▶ $\text{num1} + \text{num2} * \text{num4}$

Changing the values of variables

- Sometimes we need to adjust the values of our numerical variables (integers or floats)
- Of course we could do this with the help of Re-Assignments but sometimes we need to increase / decrease the value of a variable just by one or with the help of a result where we don't know the value beforehand
- In these cases a simple Re-Assignment doesn't make sense
- Instead we use a slightly more complex Re-Assignment that uses our variable on both sides of the equal sign:

```
number1 = number1 + 1
```

- In Python we have short-cuts for these kinds of Re-Assignments

Instruction	Variant 1	Variant 3
Increase number1 by 1	<code>number1 = number1 + 1</code>	<code>number1 += 1</code>
Decrease number1 by 1	<code>number1 = number1 - 1</code>	<code>number1 -= 1</code>
Three times number1	<code>number1 = number1 * 3</code>	<code>number1 *= 3</code>
number1 divided by 3	<code>number1 = number1 / 3</code>	<code>number1 /= 3</code>
Increase number1 by 3	<code>number1 = number1 + 3</code>	<code>number1 += 3</code>
Decrease number1 by 3	<code>number1 = number1 - 3</code>	<code>number1 -= 3</code>
Increase number1 by number2	<code>number1 = number1 + number2</code>	<code>number1 += number2</code>

Please keep in mind that these methods change the value of your variable permanently

- If you want to keep the original value of your variable than you would use something like:

```
Result1 = Var1 + 1
```

- If you want to adjust the value of your variable than you use:

```
Var1 += 1
```

or

```
Var1 = Var1 + 1
```

- This is the pythonic way to change the value of your variables

- Write a program that defines two numerical variables (one integer and one float)
- Print out their values
- Afterwards adjust the values of these two variables according to the table below
- Print out the final value of both variables as well

Integer variable	Float variable
Increase by 4	Decrease by 8
Multiply by 3	Divide by 3.7
Decrease by 25	Increase by 11.78
Divide by 5	Multiply by 6.6
Increase by -15	Decrease by -10.4
Multiply by -7	Divide by -8.9

- Variables are placeholders for actual values
- Everytime a program reads the name of a variable it's replaced with the correct value
- We use variables to be able to use the same value over and over again
- We can use variables for mathematical operations
- We can change the values of variables
- We can use `type()` to determine the data type of a variable
- We can use casting functions to change the data type of a variable

- Use variables as soon as you need to work with a specific value more than once
- Try to avoid spamming your script with a large amount of variables
 - Make use of the shortcuts for adjusting values as much as possible
- Print out your variables as much as possible to see how their value behaves, maybe include the type of them as well

What are datastructures?

- Until now we worked with variables that contained a simple value, the so-called primitives
- But we can also collect multiple values in one variable with the help of datastructures
- There are many different datastructures available:
 - ▶ lists
 - ▶ dictionaries
 - ▶ sets
 - ▶ tuples
 - ▶ DataFrames
 - ▶ Series
 - ▶ ndarrays
 - ▶ SeqRecords
 - ▶ Bio.PDB.Structures
 - ▶ ...

- Each datastructure has its own rules and functions that are associated with it
- Some datastructures are built-in, that means they are part of the standard python installation, while others are part of external modules
- At this point in time we will discuss only the built-in ones
- External ones will be discussed once we talk about modules or in later parts of the further education
- The values inside our datastructures are called elements of them and many datastructures allow us to access these elements in specific ways

- We already used functions like `print()` or `type()` and there are also functions that work only with a specific datastructure
- These kind of functions are called methods
- There are list methods, dictionary methods and so on
- Here we will talk about some of these methods that can come in handy later

Lists

- Numbered collection of values
- Only the list has a name
- Elements are addressed with [] and a number (the index)
- Index starts at 0
- Lists are:
 - ▶ ordered: that means the order of the elements is always the same
 - ▶ changeable: that means we can replace elements, add or remove elements
 - ▶ indexed: each element can be accessed by using a specific index
- Lists allow duplicate elements as well as elements of different data types at the same time

```
weekdays = ["Monday", "Tuesday", "Wednesday", "Thursday",  
            "Friday", "Saturday", "Sunday"]
```

Graphical Representation:

Index	0	1	2	3	4	5	6
Element	"Monday"	"Tuesday"	"Wednesday"	"Thursday"	"Friday"	"Saturday"	"Sunday"

Length of the list: 7

- There is no limit to the amount of elements we can place inside a list
- Of course we are able to generate lists with numbers as well:

```
number_list = [1,5,6,7,2,500]
```

```
float_list = [5.6,7.8,1.2,3.4,2.5]
```

- Furthermore we can generate lists that contain mixed data types

```
mixed_list = ["Hello",500,7.8,"World",True]
```

- That means the general syntax for defining a list is as follows:

```
listname = [ element1, element2, ..., elementn ]
```

- Of course the list name is freely choosable by you

- Write a Program that creates at least five different lists
- Two of them should contain only strings and two of them should contain only numbers
- The fifth one should mix strings and numbers

- Sometimes we need to know how many elements our lists contain
- The number of elements of a list is called the length of a list
- You can check for the lengths of lists with the help of the len() function

```
len(weekdays)
```

- We can use this function in combination with print to show the length of a list in the terminal:

```
print(len(weekdays))
```

- We can also use it to assign the result to a new variable:

```
list_length = len(weekdays)
```

- As you will see later the built-in function len() is not limited to lists only
- However the result of this function is always an integer number representing the amount of elements in the corresponding data structure

- With the help of the function range you can create lists containing a sorted range of numbers
- ATTENTION: range is a generator producing a range object
- If you want to create a list with range it has to be casted to list with the function list():

```
Numbers1 = list(range(16))
```

```
Numbers2 = list(range(5, 26))
```

```
Numbers3 = list(range(0, 21, 2))
```

- I said that the built-in function produces a so-called range object
- An object refers to an instance of a specific data type or type of data structure
- A variable `number1` declared as seen below for example would be an integer object:

```
number1 = 10
```

- A variable called `list1` declared as seen below would be a list object:

```
list1 = [1,5,10]
```

- There are many different types of objects depending on the functions, data structures and values you use in your script

- The built-in function range() takes 1, 2 or 3 arguments to create an assorted collection of numbers
- Using range with 1 argument means we specify the not included endpoint of a range of numbers that starts with the number 0:

`list(range(10))` ➔ `[0,1,2,3,4,5,6,7,8,9]`

- Using range with 2 arguments means we specify first the start point and second the not included endpoint of a range of numbers:

`list(range(5,15))` ➔ `[5,6,7,8,9,10,11,12,13,14]`

- Using range with 3 arguments means we specify first the start point, second the not included endpoint and the stepsize of a range of numbers

`list(range(5, 16, 2))` → `[5, 7, 9, 11, 13, 15]`

- range() only accepts integer values for its arguments, that means the elements in our list later on are also all integers
- If you need a sorted list of floating point numbers than you would need to use the module NumPy and it's arange() function (We will talk about this later)

- Using range with one argument in form of a negative number leads to an empty range

```
list(range(-10))
```

→ []

- When working with range() and two arguments the start point needs to be smaller than the end point, otherwise you produce an empty range

```
list(range(10, 0))
```

→ []

- However when using three arguments the start point can be bigger than the end point as long as you use a negative step size

```
list(range(10, 0, -1))
```

→ [10,9,8,7,6,5,4,3,2,1]

- Write a program that generates four lists with different lengths by using the range() function
- The program should generate a list that doesn't start at 0
- The program should generate two other lists that skip some numbers (three arguments for range)
- Let the program print out the generated lists as well as the corresponding lengths

- Now that we created our collections of elements, we want to work with them
- To be able to do this we have to access the different elements separately
- This can be done with the help of indices
- Indices are whole numbers that specify which element you want to access in certain datastructures
- They are always used with the following general syntax:

```
listname[index]
```
- The counting for indices starts at 0, that means the first element of a list has the index 0, the second element the index 1 and so on

- For example `weekdays[2]` would hand us back the element at index 2 of our list `weekdays` (which would be the third element)
 - This element has the value „Wednesday“
- Be careful when working with indices, since we can only use indices that are available to the corresponding list
 - That means a list with 5 elements has only the indices 0, 1, 2, 3 and 4 available
 - Using an index of 10 on this list for example would lead to an `IndexError` with the message `list index out of range`
 - This means we used an index that was not available to our list
- You can't access multiple elements at the same time

- Use your lists from Exercise 9 to access single elements of them
- Print out 2 different elements for each list
 - ▶ Indices 0 & 3 for the first list
 - ▶ Indices 7 & 10 for the second
 - ▶ Indices 2 & 5 for the third
 - ▶ Indices 12 & 4 for the fourth
 - ▶ Indices 1 & 6 for the fifth list

- We saw how to use indices to access elements in our lists
- We can also use the index in an assignment to help us with replacing the value of an element in our list
- By using the code below we can change the value of the element at index 3 in our list weekdays to the value of „Hello World“

```
weekdays[3] = „Hello World“
```

- Again we can only use indices that are available to our list here
- We can freely change the data types of the elements in our list during this process:

```
weekdays[2] = 500
```

- Use your lists from Exercise 9 and replace the values of at least two elements in each of them
- Please print out the lists before and after you replaced the values of the elements in them
- Replace the elements as follows:
 - ▶ at index 1 & 3 for the first list
 - ▶ at index 6 & 8 for the second
 - ▶ at index 0 & 3 for the third
 - ▶ at index 10 & 9 for the fourth
 - ▶ at index 3 & 5 for the fifth list

- As you know you can access a single element of the list `numbers=[1,2,3,4,5]` as follows `numbers[0] = 1` or `numbers[3] = 4`
- That means accessing elements from the start of your lists is very straight forward
- For accessing elements from the end of your lists there is also a relative straight forward method in Python
- This method is the usage of negative indices instead of positive numbers

- In Python the index -1 always stands for the last entry:
 - `numbers[-1] = 5`
- The index -2 for the second to last
 - `numbers[-2] = 4`
- and so on
- Again you're limited to the available indices of your list, that means for a list with 5 elements the lowest index we can use is -5

- The logic behind negative indices can be explained with an example
- Take the following list:

```
numbers = [1, 2, 3, 4, 5]
```

- Now we can determine the length of this list:

```
length = len(numbers) → the value here is 5
```

- We know that indexing starts at 0 and that means the highest possible index for our list is 4
- As you can see the highest possible index of 4 can also be calculated as follows:

```
max_index = length - 1 → max_index = len(numbers) - 1
```

- If we now use our calculated biggest index on our list we get:

```
numbers[max_index]
```

→ `numbers[length - 1]`

→ `numbers[len(numbers) - 1]`

- This is where negative indices originally stem from and this also explains why the lowest index is -5:

`len(numbers) - 5` → This is 0

- Use your lists from exercise 10 and print out the last element (-1), as well as the third to last (-3) and fifth to last element (-5) of them using negative indices

- Sometimes we need only a small part of our list containing several elements of it
- In these cases we can use so called slices that can be generated with the help of indices and a colon “:”
- Slicing is used to return parts of a list and can be used as follows

```
listname[first_index:last_index]
```

➔ It cuts before the first index and before the last index (not included in new list)

- Take the following list:

```
things = [0, 'Fred', 2, 'S.P.A.M.', 'Rocking', 42, "Jack", "Jill"]
```

- We can generate different slices / sublists of it by using the following syntax:

```
things[1:6]          → ['Fred', 2, 'S.P.A.M.', 'Rocking', 42]
```

```
things[2:4]          → [2, 'S.P.A.M.']
```

```
things[4:7]          → ['Rocking', 42, "Jack"]
```

- Keep in mind that the original list Things is not changed during this process
- In most cases you assign the sublist to a new name:

```
sublist1 = things[2:4]
```

- Write a program that generates two different lists with contents of your choice
- Define four numbers
- Use them to slice your lists
- What happens if you use numbers that are bigger than the size of your lists?

- Python allows you to leave out either the `start_index` or the `end_index` of a slice to generate very specific slices
- If we leave out the `start_index`, slicing starts before the index 0 and ends before the `end_index`

`numbers[:3]` → Generates a sliced list that contains the first three elements of numbers

- If we leave out the `end_index`, slicing starts before the `start_index` and goes on until the end of the list

`numbers[3:]` → Generates a sliced list that starts at the fourth element of numbers and includes all elements afterwards

- Write a program that generates a list with the help of the range function
- Create a slice of the list that contains the first 5 numbers and one that contains the first 8 numbers
- Create another slice that starts at index 6 and one that starts at index 10
- Print out the sliced lists

- In Python we have functions that are able to work with lists
- We already saw some simple examples like `len()`, `print()` or `type()`
- There are many more functions we can use in a similar way
- Sometimes functions that can use lists as arguments are called list functions
- However they are NOT unique to lists, and instead can use other types of arguments as well

- Furthermore there are functions that work directly on our lists, so we don't need to use the list as an argument for our function
- Instead we use a point notation which tells python which function should be executed on our list:

```
our_list.function_to_execute()
```

- Functions we use like this are called (list) methods since they directly work on the (list) object we specified before the point
- Different kind of data structures have different kinds of methods

- In Python lists can be elongated with the help of different methods
- The most general way is by adding elements to the end of a list with the methods `append` or `extend`
- These methods have different usages depending on your needs (see next slides)
- There is also another method that allows us to add new elements anywhere in our list, but we will talk about this a bit later

- The method `append` takes any argument and adds it unchanged to the end of the corresponding list
- We need to invoke it on a list we defined beforehand and we permanently change our list when using it
- Furthermore we can NOT use it as argument of another function or in an Assignment
- The general syntax is as follows:

```
listname.append(element_to_add)
```
- The `element_to_add` can be anything (a number, a text, another list)

- The method `extend` takes only arguments that are „iterable“, that means datastructures and adds each element of them to the end of the list
- We need to invoke it on a list we defined beforehand and we permanently change our list when using it
- Furthermore we can NOT use it as argument of another function or in an Assignment
- The general syntax is as follows:

```
listname.extend(list_of_elements_to_add)
```
- The `list_of_elements_to_add` can be any kind of datastructure (a list, a tuple etc.)

- Imagine the two lists listA and listB down below:

```
listA = [1, 2, 3]
```

```
listB = [4, 5, 6]
```

- Now we can add the number 6 to the end of listA as follows:

```
listA.append(6)                ➔ [1, 2, 3, 6]                ➔ length: 4
```

- We can also add a String unchanged to the end of listA using append as follows:

```
listA.append("Hello")          ➔ [1, 2, 3, 6, "Hello"]          ➔ length: 5
```

- We can also add the elements of listB to the end of listA using extend as follows:

`listA.extend(listB)` → `[1, 2, 3, 6, "Hello", 4, 5, 6]` → `length: 8`

- We can also do the reverse, that means adding the elements of the now long listA to the end of listB with extend as follows:

`listB.extend(listA)` → `[4, 5, 6, 1, 2, 3, 6, "Hello", 4, 5, 6]` → `length: 11`

- Please be aware that extend can NOT be used with single numbers like we saw for append

- Write a Program that generates four different lists
- Append a number of your choice to first one and a different number to the second one, e.g. `list1.append(1000)`
- Extend the first list with the help of the third list and the second one with the help of the fourth list, e.g. `list1.extend(list3)`

- You can add elements to specific points in your list with the help of the insert method()
- General Syntax:

```
List.insert(index,value)
```

- So to add the number 20 at index 3 to the list down below we could use the following code:

```
Mylist = [0,1,2,3,4,5,6]
```

```
Mylist.insert(3,20)
```



```
[0,1,2,20,3,4,5,6]
```

- The insert() method is very robust
- If we use an index that is bigger than the length of our list, the new element is simply added to the end of our list:

`Mylist.insert(50, 300)` → `[0, 1, 2, 20, 3, 4, 5, 6, 300]`

- On the other hand if we use the index 0 for insert we would enter the new element to the beginning of our list:

`Mylist.insert(0, 30)` → `[30, 0, 1, 2, 20, 3, 4, 5, 6, 300]`

- We can even use negative indices, e.g -2 in combination with the insert() method
- In these cases we insert the new element before the specified position, that means -1 puts something before the last element for example:

`Mylist.insert(-1, 50)` → `[30, 0, 1, 2, 20, 3, 4, 5, 6, 50, 300]`

- Again insert() is very robust, that means if we use something like -50 for example, the new element would simply be added at the beginning of our list

`Mylist.insert(-50, 9)` → `[9, 30, 0, 1, 2, 20, 3, 4, 5, 6, 50, 300]`

- Write a program that generates three lists of different sizes and print them out
- Afterwards insert values into these lists at points of your choosing

- You can delete elements from lists with the `remove()` method
- The general syntax is as follows:

```
listname.remove(element_to_remove)
```

- The argument `element_to_remove` needs to be an element present in your list, if not you will encounter a `ValueError` with the message `list.remove(x): x not in list`
- `Remove` only takes one argument so you can remove only one element at a time
- Let's say you have a list of fruits as shown below and now you want to delete the element "banana" from it, you can do so as follows

```
fruitlist = ["apple", "banana", "orange"]  
  
fruitlist.remove("banana")
```

- The method `remove()` only deletes the first appearance of the element you want to delete:

```
list1 = [10, 5, 10, 4, 10, 3]
```

```
list1.remove(10) → list1 = [5, 10, 4, 10, 3]
```

- So if you want to delete the second appearance of the element as well then you have to use `remove` again:

```
list1.remove(10) → list1 = [5, 4, 10, 3]
```

- Write a program that generates at least three different lists
- Print these lists out
- Let the program remove one element from each list
- Print out the new lists

- Similar to the methods `append()`, `extend()`, `remove()` and `insert()` we have additional methods that work on lists and use the point notation
- Some of these methods return single values, while others return nothing
- Functions that return nothing and have to be used without an Assignment:
 - ▶ `list.reverse()`: reverses the order of the list
 - ▶ `list.sort()`: sorts the list in ascending order
 - ▶ `list.clear()`: removes all elements from the list
- These functions work in-place and therefore permanently change our list as soon as they are executed

- Functions that return a single value we can assign to a new variable:
 - ▶ `list.index(X)`: returns the first appearance of a particular value
 - ▶ `list.count(X)`: returns the number of elements with the required value
- Functions that return a list we can assign to a new variable
 - ▶ `list.copy()`: creates a duplicate of the list

- Aside from functions that work on a specific list with the point notation we also have functions that take a list as argument and hand us back a single value:
 - `max(list)`: returns the maximum value
 - `min(list)`: returns the minimum value
 - `sum(list)`: returns the sum of all elements of a list containing only numerical values
- These functions work similar to `len()`
- While `sum()` only works on lists with numerical values, `min()` and `max()` work also on lists with strings
- BUT only if our lists contain only strings, a list that contains mixed elements, numbers and text, can NOT be used here

- Use your lists from Exercise 9, except the mixed one
- Determine the elements with the highest and lowest values in them
- Calculate the overall sum of all elements for the two lists containing numbers
- Afterwards sort all four lists in ascending order and print them out

Dictionaries

- Dictionaries are the second type of built-in datastructure I want to discuss here
- They differ from lists in many aspects
- Instead of just collecting elements dictionaries work with key/value pairs to save elements
- The keys should be strings that are used to find the corresponding values, which can be everything (ints, floats, strings etc.)
- All keys of a dictionary have to be unique, that means we can NOT have the same key twice

- There are two ways of defining a dictionary in Python
- The first version is by defining an empty dictionary first and then fill it with data step by step
- To define an empty dictionary we need curly brackets { } and an assignment to a name of our choice
- For example we can define an empty dictionary called numbers as follows:

```
numbers = {}
```

- Now we can add an element to our empty dictionary numbers as follows:

```
numbers["Bennet"] = 56789123
```

- In the example below we assign the value 56789123 to the non-existing key "Bennet" in our dictionary
- Like this we create the new key/value pair of "Bennet"/56789123
- We can now fill our dictionary by adding more key/value pairs to it

```
numbers["Gundolf"] = 1234567
```

```
numbers["Christoph"] = 3456789
```

- Another way of defining a dictionary would be to directly specify the key/value pairs in the definition of the dictionary
- Again we need curly brackets { } and then we can specify the key value pairs of our dictionary inside these brackets:

```
numbers = {"Bennet":56789123, "Gundolf":1234567, "Christoph":3456789}
```

- Each key/value pair has to be written as follows inside the brackets:
 - key:value
- Different key/value pairs are then comma separated inside the brackets as shown in the example above

- There are two dictionary methods that can come in handy when working with dictionaries
- The method `keys()` returns a `dict_keys` object containing all keys of the dictionary `numbers` and can be called as follows:

```
numbers.keys()
```

- The second method is called `values()` and returns a `dict_values` object containing all values of the dictionary `numbers`:

```
numbers.values()
```

- Be aware that these are methods/functions so we need the empty brackets

- Accessing values in our dictionaries is similar to accessing elements in lists
- We need the name of our dictionary, square brackets [] but this time we use the key instead of an index
- For example to access the value associated with the Key “Bennet” in our dictionary numbers we can use:

```
numbers["Bennet"]
```

- Since we use a syntax that is very similar to lists, it may be good idea to always use strings as your keys to be able to differentiate between dictionaries and lists in an easier manner later on

- We saw earlier how to define our dictionaries with the help of square brackets [] and assigning values to new keys
- If we use the same syntax again but with a different value we replace the original value in our dictionary
- For example we can replace the value associated with the key “Bennet” (56789123) in our dictionary numbers as follow:

```
numbers["Bennet"] = 500
```

- Now the key “Bennet” is associated with the new value 500

- We can remove key/value pairs from our dictionaries with the function `del`
- For example to remove the key/value pair of “Bennet”/500 from our dictionary `numbers` we could use:

```
del (numbers["Bennet"])
```

- This only removes a key/value pair from a dictionary as long as the corresponding key exists, if not you will get back an error message `KeyError: ...`
- We can only remove complete key/value pairs, that means we are not able to remove only a value of a key for example

- Write a program that generates a simple dictionary consisting of at least three pre-defined entries with biological or biochemical background
- Print out the dictionary and afterwards insert a new key/value pair into your dictionary and remove a different key/value pair
- Print out the remaining dictionary

Tuples

- Tuples are a third form of datastructures that can be generated in Python
- They are ordered, unchangeable and indexed
 - ▶ Ordered means that you can't change the order of the elements
 - ▶ Unchangeable means that you can't add/remove elements from the tuple or replace elements
 - ▶ Indexed means we can access single elements of our tuple just like we did with lists

- They are generated with the help of round brackets:

```
Mytuple = ("apple", "banana", "orange", "melon", "peach")
```

- If you want to generate a tuple with just one element you always need to add a comma after this element so that Python can interpret it as a tuple

```
Mytuple2 = ("apple",)
```

- In most cases we use tuples in combinations with our own functions
- Since we can define functions that hand us back several values we can use tuples to collect these values for example (More about this when we talk about functions)
- Another usage is in combination with certain functions from external modules, like the mysql-connector-python module for example (more about this in our second module)

- As mentioned earlier we can access the elements in our tuples by using indices, just like we could with lists
- To get access to the element at index 0 in our tuple Mytuple we can use:

```
mytuple[0]
```

- We can also use slices to produce smaller tuples containing only certain elements of our original tuple:

```
mytuple[1:3]
```

- Write a program that creates four tuples of different lengths
- Print out the tuples
- Afterwards display one element from each tuple
- Try to generate a tuple with duplicate values.

Sets

- Sets are the last type of datastructure that is included in the standard Python package
- Sets are unordered, changeable and can't be indexed
 - ▶ Unordered means that the order of the values can change on different calls of our program
 - ▶ Changeable means that you can't change the elements in the set but you can add or remove elements from it
 - ▶ Not indexed means we aren't able to access single elements in our sets

- They are also created with curly brackets but you can't create an empty set:

```
Myset = {"apple", "banana", "orange", "melon", "peach"}
```

- We can however create sets with just one element:

```
Myset = {"apple"}
```

- Seeing the different properties of sets, they don't seem to be very useful and in most cases you wouldn't collect values in sets to be honest
- However sets have one big advantage and that is that they don't allow duplicate values
- So we can make use of this property when filtering lists for example
- Python contains a casting function that allows us to change a list to a set:

`list1 = [5, 6, 5, 6]` ➔ contains the elements 5, 6, 5 and 6

`set1 = set(list1)` ➔ contains the elements 5 and 6

- Changing from a list to set is always possible and in the process all duplicate values are removed from the list

- Afterwards we can change the set back to a list with the casting function `list()` we saw when working with `range()`:

```
list2 = list(set1)
```

➔ contains the elements 5 and 6

- The most pythonic way to do something like this is to write everything in one line and re-assign the new list to the same name as the original list:

```
list1 = list(set(list1))
```

- Now our `list1` is converted to a set first, this set is then converted to a list and this list is then re-assigned to our variable `list1`

- Write a program that creates four sets of different lengths
- Print out the sets

Nesting datastructures

- We learned that lists, dictionaries, tuples and sets are collections of multiple elements / values
- In Programming we are able to not only collect singular values like numbers or strings in lists or dictionaries but also other datastructures
- This process is called nesting of datastructures
- For now we will look at only “two-dimensional” datastructures, that means lists that contain other lists, which then contain numbers and/or strings for example
- There is no “dimensional” limitation to the process of nesting datastructures so we could generate “ten-dimensional” lists as well

- Contain two or more running indices, depending on how many lists we nest into each other
- Are the least standardized, but we are able to use our already known list methods on them
- `[[], []]` – could be used to create an empty two-dimensional list in Python
- `[[1, 2, 3], [4, 5, 6]]` – could be used to create a two-dimensional list which contains two sublists with three elements each in Python

- We can create a two-dimensional list with the following syntax:

```
multi_list1 = [[1,2,3],[4,5,6]]
```

- The list `multi_list1` contains two elements which are also lists, these are then called “sublists” of `multi_list1`, while `multi_list1` is called the outer list
- We can access these “sublists” by using indices on our outer list:

```
multi_list1[0]           ➔ [1,2,3]
```

```
multi_list1[1]           ➔ [4,5,6]
```

- We can access the elements of the sublists by chaining together the indices of our outer list and the sublists:

`multi_list1[0][0]` ➔ 1

`multi_list1[1][2]` ➔ 6

- If you want to use a list method on one of your “sublists” you always need the name of your outer list and the corresponding index
- For example if you want to add an element to the end of the second “sublist” of our nested list `multi_list1` you can use:

```
multi_list1[1].append(20)           ➔  [[1, 2, 3], [4, 5, 6, 20]]
```

- Like this you can run list methods on the “sublists”

- Write a program that creates at least three different two-dimensional lists with different sizes
 - ▶ One of them should contain 4 sublists with 4 elements each
 - ▶ One should should contain 3 sublists with 6 elements each
 - ▶ One should should contain 6 sublists with 3 elements each
- Print out two elements of your choice (use the indices you want) of each two-dimensional list

- Recap:
 - ▶ Dictionaries have keys and values
 - ▶ Keys can be strings or numbers but strings are preferred
 - ▶ Keys point to values
 - ▶ Values can be any type of variable, including lists or dictionaries, that can also contain lists or dictionaries and so on
 - ▶ Sounds Scary?

- Similar to how we could nest lists into each other, we can also nest dictionaries into each other:
 - ▶ `multi_dict1 = {"Key1":{"In1":2,"In2":5}, "Key2":{"In3":10,"In4":25}}`
- `multi_dict1` contains two key/value pairs and each key is associated with another dictionary:
 - ▶ `multi_dict1["Key1"]` → `{"In1":2,"In2":5}`
 - ▶ `multi_dict1["Key2"]` → `{"In3":10,"In4":25}`
- Here `multi_dict1` is our outer dictionary and the two other dictionaries are than our “subdictionaries” of `multi_dict1`

- When creating a nested dictionary the sub dictionaries are independent from each other and thus they can contain the same keys:
 - ▶ `multi_dict1 = {"Key1":{"In1":2,"In2":5}, "Key2":{"In1":10,"In2":25}}`
- To access the value associated with a key of a “subdictionary” we need to chain together the keys, similar to what we did with the indices on our multi-dimensional lists:
 - ▶ `multi_dict1["Key1"]["In2"]` → 5
 - ▶ `multi_dict1["Key2"]["In2"]` → 25

- Write a program that creates a complex dictionary in form of a protein collection
- Each key of the dictionary „proteins“ should be associated with another dictionary as value in which the name, the length of the protein, the function and the location are collected
- The final dictionary should contain information of at least two proteins
- Print out the final dictionary at the end

Definitions

- The **Declaration** of a variable refers to the creation and naming of a variable
- We use **Assignments** to specify the value of a variable
- The **Value** of a variable is the number or text it contains
- **Re-Assignment** of a variable is the process of assigning a new value to an already existing variable
- The **Data Type** of a variable refers to an attribute of our variable that tells the computer how to deal with it
- A **Print Out** is the process of displaying something in the terminal
- The process of converting the value of a variable to another data type is called

Casting

- **Lists** are simple datastructures that can be used to collect multiple values in the same variable
- An **Index** is a whole number and is used to access an element inside specific datastructures
- Each entry in a datastructure is called **Element**
- **Slicing** is the process of generating a smaller sublist using (two) indices and a :
- A **List Method** is a function that works directly with lists and only with lists
- We can access elements in dictionaries using **Keys**
- Each key of a dictionary is associated with one value to form a **Key/Value Pair**

- The term **Nesting** refers to the act of collecting datastructures inside other datastructures, but can also be used to describe the combination of if-statements and/or loop inside each other
- If a list is an element of a datastructure it's called **Sublist**
- When a list contains sublists it's called an **Outer List**
- A dictionary inside another dictionary is called **Subdictionary** while a dictionary that contains other dictionaries is called **Outer Dictionary**

1. [https://en.wikipedia.org/wiki/Variable_\(computer_science\)](https://en.wikipedia.org/wiki/Variable_(computer_science)) (Last edited: 03.12.2022, Last visited: 04.01.2023)
2. https://en.wikibooks.org/wiki/Non-Programmer%27s_Tutorial_for_Python_3 (Last edited: 05.09.2022, Last visited: 02.01.2023)
3. <https://realpython.com/lessons/duck-typing/> (Last visited: 02.01.2023)
4. <https://stackoverflow.com/questions/4205130/what-is-duck-typing> (Answer edited: 06.03.2023, Last visited: 05.04.2023)
5. <https://realpython.com/python-data-types/#built-in-functions> (Last visited: 05.04.2023)
6. <https://realpython.com/python-print/> (Last visited: 05.04.2023)
7. <https://realpython.com/python-f-strings/> (Last visited: 05.04.2023)
8. <https://www.geeksforgeeks.org/type-casting-in-python-implicit-and-explicit-with-examples/> (Last edited: 30.01.2021, Last visited: 05.04.2023)
9. <https://docs.python.org/3/tutorial/floatingpoint.html> (Last edited: 04.04.2023, Last visited: 05.04.2023)
10. <https://realpython.com/python-input-output/> (Last visited: 05.04.2023)

11. https://www.w3schools.com/python/python_tuples.asp (Last visited: 05.04.2023)
12. https://www.w3schools.com/python/python_sets.asp (Last visited: 05.04.2023)
13. https://en.wikipedia.org/wiki/Binary_number (Last edited: 25.02.2024, Last visited: 05.03.2024)
14. <https://en.wikipedia.org/wiki/Octal> (Last edited: 30.01.2024, Last visited: 05.03.2024)
15. <https://en.wikipedia.org/wiki/Hexadecimal> (Last edited: 04.02.2024, Last visited: 05.03.2024)